TASK 01: UNDERSTAND THE PROBLEM AND IDENTIFY THE DATA

Course: CSA08 – Python Programming
Assignment: Payment Calculator for a Village Milk Co-operative
Marks: 4
Bloom's Level: K4 – Analyse

Student Name: B.Sri vyshnavi
Register Number:192412488

1. Introduction

The Ksheera Village Milk Producers' Co-operative collects milk from approximately 300 farmers twice a day. The quantity of milk, fat percentage and SNF percentage are recorded for every farmer.

The objective is to develop a Python program that calculates the correct milk rate per litre, individual payment and ten-day payment total.

2. Identify the Inputs and Outputs

Inputs

Input	Data type	Description
Farmer ID	String	Unique identification of farmer
Litres	Float	Quantity of milk supplied
Fat percentage	Float	Fat content of milk
SNF percentage	Float	Solids-not-fat content
Shift	String	Morning or Evening

Outputs

1. Rate per litre.
2. Amount payable for each milk entry.
3. Total payment for ten days.

Explanation

The inputs are required to calculate the milk price based on its quality. The outputs help the co-operative determine the correct payment for each farmer.

3. Named Constants

The following values should be declared as constants:

Constant	Value	Purpose
BASE_RATE	30.00	Standard price per litre

STANDARD_FAT	3.5	Standard fat percentage
STANDARD_SNF	8.5	Standard SNF percentage
FAT_STEP_VALUE	0.40	Price adjustment for every 0.1% fat
SNF_STEP_VALUE	0.30	Price adjustment for every 0.1% SNF
FAT_STEP_SIZE	0.1	Fat percentage step
SNF_STEP_SIZE	0.1	SNF percentage step
MIN_FAT	2.0	Minimum permitted fat
MAX_FAT	7.0	Maximum permitted fat

Constants are used because these values remain unchanged during a normal program execution. If the co-operative changes the rates, the values can be updated in one place.

4. Data Types and Their Justification

Integer (int):

- Number of farmers.
- Number of entries.
- Number of quality steps.

Float (float):

- Milk quantity.
- Fat percentage.
- SNF percentage.

String (str):

- Farmer ID.
- Shift name.

Decimal:

- Payment amounts and financial totals, to avoid floating-point errors.

Decimal is preferred for accurate monetary calculations.

5. Validation Rules

The program must satisfy the following conditions:

1. Milk quantity must be greater than zero.
2. Fat percentage must be between 2.0% and 7.0%, inclusive.
3. Farmer ID must not be empty.

4. Shift must be either Morning or Evening.
5. Fat and SNF must contain valid numerical values.
6. Invalid entries must not be included in payment calculations.
7. Payment must be rounded to two decimal places.

Note: The assignment does not specify an SNF minimum and maximum range. Therefore, the actual analyser limits should be confirmed before implementing a production system.

6. Assumptions

1. Every farmer has a unique farmer ID.
2. Each farmer can supply milk in both morning and evening shifts.
3. The standard milk rate is Rs. 30 per litre.
4. Every 0.1% change in fat adjusts the rate by Rs. 0.40.
5. Every 0.1% change in SNF adjusts the rate by Rs. 0.30.
6. Payments are calculated for every ten-day period.
7. The nearest whole quality step is used for the original rate calculation.

7. Alternative Concept and Complexity Analysis

Separate Variables vs List or Tuple

Method 1: Separate Variables

If separate variables are used for every farmer, the program needs variables such as:

- litres1, fat1, snf1
- litres2, fat2, snf2
- litres3, fat3, snf3

For 300 farmers, 900 separate variables would be required for these three measurements.

This makes the program difficult to maintain and update.

Method 2: List or Tuple

Instead, each farmer's entry can be stored together:

```
entries = [
   (10, 3.5, 8.5),
   (8.5, 4.2, 8.7),
   (12, 3.1, 8.3)
]
```

The program can process all entries using a loop.

Complexity Comparison

Factor	Separate Variables	List/Tuple
Processing time	O(n)	O(n)
Storage	O(n)	O(n)
Maintenance	Difficult	Easy
Scalability	Poor	Good
Loop processing	Difficult	Simple

Both approaches require O(n) storage when all entries are retained. However, lists and tuples are easier to manage and scale.

8. Conclusion

A list or tuple-based design with suitable data types, named constants and validation rules is recommended for the milk payment calculator.

It reduces code duplication, improves readability and allows the program to process increasing numbers of farmers efficiently.